

校园互助服务平台—详细设计文档（整合版）

阶段：Phase 3 详细设计

版本：2.1

最后更新：2026-06-16

1. 引言

本文档整合了校园互助服务平台详细设计阶段的所有产出，包括类图设计、API 规范、数据库设计（ER 图 + 建表 SQL）以及实际工程中的设计模式应用。本文档内容与项目实际代码一致（基于 Flyway V1-V14 数据库迁移和 MyBatis-Plus 实体模型）。

各详细内容请参见对应的独立交付物文件。

2. 类图设计

2.1 架构决策：单表 + 类型鉴别器

实际实现采用扁平实体模型（11 个 MyBatis-Plus POJO，无继承关系），而非最初规划中的类表继承体系（BaseOrder $\square$ PaidOrder $\square$ 具体类型）。核心需求实体 Demand 通过 type 鉴别器列（errand / trade / team / lost_found / study / other）和 JSON attributes 列实现六种需求类型的统一存储。

选择理由： - MyBatis-Plus 不支持 JPA 的 @Inheritance 注解，单表映射最自然 - 需求列表查询无需跨多表 JOIN 或 UNION - 新增类型无需 ALTER TABLE，仅需应用层扩展验证规则 - 符合项目”简单可维护”的工程目标

2.2 实体清单

模块	实体	对应表	说明
用户认证	User	user	用户身份与认证
	PrivacyProfile	privacy_profile	匿名模式与虚拟昵称
	UserAccount	user_account	积分余额与信誉分
需求业务	Demand	demand	需求（6 种类型，单表）
	TeamMember	team_member	组队成员与审批
聊天社交	Conversation	conversation	聊天会话
	Message	message	聊天消息
	Notification	notification	系统通知
	Evaluation	evaluation	交易评价（1-5 星）
积分系统	PointsTransaction	points_transaction	积分流水账本
	DailyCheckin	daily_checkin	每日签到记录
收藏与举报	Favorite	user_favorite	需求收藏/书签
	Report	report	内容举报记录
成就徽章	UserBadge	user_badge	已获得徽章记录
	WornBadge	worn_badge	当前佩戴徽章

2.3 核心关系

- **User $\square$ Demand (1:N $\times$ 2)**：一个用户可发布多个需求，也可接取多个需求
- **User $\square$ UserAccount (1:1 组合)**：用户注册时自动创建账户，随用户删除而级联删除

- **User** $\square$ **PrivacyProfile** (1:1 组合): 用户注册时自动创建, 随用户删除而级联删除
- **Demand** $\square$ **TeamMember** (1:N): 组队需求包含多条成员记录 (含 PENDING/JOINED/REJECTED 状态)
- **Demand** $\square$ **Conversation** (1:0..1): 需求被接取后可创建专属聊天会话
- **Conversation** $\square$ **Message** (1:N 聚合): 会话包含多条消息, 消息可独立于会话存留
- **Demand** $\square$ **Evaluation** (1:0..2): 每次交易最多产生 2 条评价 (发布者 $\square$ 接单者互评)
- **User** $\square$ **Favorite** (1:N): 用户可收藏多个需求
- **User** $\square$ **Report** (1:N): 用户可提交多条举报
- **User** $\square$ **UserBadge** (1:N): 用户可获得多种徽章
- **User** $\square$ **WornBadge** (1:1): 用户最多佩戴一枚徽章

完整类图 (Mermaid 格式, 含全部 15 个实体、字段和关系) 请见 类图.md。

3. API 规范设计

3.1 概述

采用 RESTful 风格, 统一前缀 /api/v1, 使用 JWT Bearer Token 认证。全局响应格式:

```
{
  "code": 200,
  "msg": " 操作成功",
  "data": {}
}
```

Common error codes: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Error.

3.2 认证机制

- 注册/登录接口无需认证 (permitAll())
- 其余所有接口需在请求头携带 Authorization: Bearer <jwt_token>
- JWT 载荷包含 sub (userId)、studentId、role
- 每次请求在 JwtAuthenticationFilter 中验证 token 有效性并检查用户封禁状态 (status=0 则拒绝)
- 管理员接口 (/api/v1/admin/**) 额外要求 ROLE_ADMIN

3.3 分页规范

分页查询使用 MyBatis-Plus Page:

参数	默认值	说明
pageNum	1	页码
pageSize	10 (demand) / 20 (transactions)	每页条数

响应中 data 包含 records (列表)、total (总数)、current (当前页)、pages (总页数)。

3.4 完整接口列表 (57 个端点)

3.4.1 用户模块—UserController (/api/v1/user)

#	方法	路径	认证	说明
1	POST	/register	无	注册 (Register-Request), 自动创建账户 + 隐私配置 + 发放注册积分 □ ApiResult<Void>
2	POST	/login	无	登录 (LoginRequest), 返回 JWT + userId/name/role/avatar □ LoginResponse
3	GET	/profile	JWT	获取当前用户完整信息 (含积分、信誉、隐私设置) □ User-InfoResponse
4	PUT	/profile	JWT	更新个人资料与隐私设置 (UpdateProfileRequest) □ UserInfoResponse
5	POST	/avatar	JWT	上传头像 (MultipartFile), 返回文件 URL □ String
6	PUT	/password	JWT	修改密码 (ChangePasswordRequest: 旧密码 + 新密码) □ ApiResult<Void>

3.4.2 需求模块—DemandController (/api/v1/demands)

#	方法	路径	认证	说明
7	POST	/	JWT	发布需求 (CreateDemandRequest), 积分类型自动冻结报酬, 组队类型自动加入发布者队长 □ DemandResponse
8	POST	/upload-image	JWT	上传需求图片 (MultipartFile), 返回文件 URL □ String
9	GET	/	JWT	需求列表 (pageNum/pageSize/type/keyword/sort) □ Page<DemandResponse>

#	方法	路径	认证	说明
10	GET	/my	JWT	我发布/接取的需求列表 (role=publisher/acceptor) □ List<DemandResponse>
11	GET	/my/team	JWT	我加入的组队需求列表 □ List<DemandResponse>
12	GET	/ {demandId}	JWT	需求详情 (含发布者信息、团队成员、匿名处理) □ DemandResponse
13	PUT	/ {demandId}/accept	JWT	接单 (仅 OPEN 状态, 组队类型需走申请流程) □ DemandResponse
14	PUT	/ {demandId}/complete	JWT	完成需求 (仅发布者, 积分从冻结转入接单者账户) □ DemandResponse
15	PUT	/ {demandId}/cancel	JWT	取消需求 (仅发布者, 退回冻结积分) □ ApiResponse<Void>
15a	PUT	/ {demandId}	JWT	编辑需求 (CreateDemandRequest, 仅发布者, 仅 OPEN 状态) □ DemandResponse

3.4.3 收藏模块—DemandController (/api/v1/demands)

#	方法	路径	认证	说明
15b	GET	/my/favorites	JWT	我的收藏列表 (分页, 含需求详情)
15c	POST	/ {demandId}/favorite	JWT	收藏需求 (幂等, 已收藏不报错)
15d	DELETE	/ {demandId}/favorite	JWT	取消收藏 (幂等)

3.4.4 组队模块—TeamMemberController (/api/v1/demands/{demandId}/team)

#	方法	路径	认证	说明
16	POST	/apply	JWT	申请加入组队 (状态 PENDING), 通知发布者
17	PUT	/applicants/{applicantId}/approve	JWT	审批通过 (仅发布者, 状态 □ JOINED)
18	PUT	/applicants/{applicantId}/reject	JWT	拒绝申请 (仅发布者, 状态 □ REJECTED)

#	方法	路径	认证	说明
19	POST	/leave	JWT	退出队伍（队长不可退出）
20	DELETE	/members/{memberId}	JWT	移除成员（仅发布者，队长不可移除）
21	GET	/members	JWT	获取已加入的成员列表
22	GET	/applicants	JWT	获取待审批的申请列表（仅发布者可见）
23	GET	/my-membership	JWT	查询当前用户在该需求中的组队状态

3.4.5 聊天模块—ChatController (/api/v1/chat)

#	方法	路径	认证	说明
24	POST	/conversations	JWT	创建或获取已有会话（含权限验证）
25	GET	/conversations	JWT	当前用户的所有会话列表（含未读数）
26	GET	/conversations/{conversationId}/messages	JWT	获取历史消息（同时标记对方消息为已读）
27	POST	/conversations/{conversationId}/messages	JWT	发送消息（支持text/image类型）
28	POST	/upload-image	JWT	上传聊天图片
29	GET	/unread-count	JWT	获取总的未读消息数

3.4.6 评价模块—EvaluationController (/api/v1/evaluations)

#	方法	路径	认证	说明
30	POST	/	JWT	提交评价（需为交易参与方，每个需求每人一次）
31	PUT	/evaluationId	JWT	修改已有评价（仅评价人）
32	GET	/demand/{demandId}	JWT	获取某需求的所有评价（含评价人姓名头像）
33	GET	/mine	JWT	获取当前用户对某需求的评价（参数demandId）
34	GET	/user/{userId}	JWT	获取某用户收到的所有评价

3.4.7 通知模块—NotificationController (/api/v1/notifications)

#	方法	路径	认证	说明
35	GET	/	JWT	当前用户的通知列表（按时间倒序）
36	GET	/unread-count	JWT	未读通知数量
37	PUT	/read-all	JWT	全部标记为已读
38	PUT	/notificationId/read	JWT	标记单条为已读

3.4.8 积分模块—PointsController (/api/v1/points)

#	方法	路径	认证	说明
39	POST	/checkin	JWT	每日签到（5~15 分，按连续天数递增）
40	GET	/checkin/status	JWT	查询今日签到状态与连续签到天数
41	GET	/transactions	JWT	积分流水明细（支持按类型筛选，分页）

3.4.9 管理模块—AdminController (/api/v1/admin)

#	方法	路径	认证	说明
42	GET	/dashboard	JWT + ADMIN	仪表盘统计（用户/需求/积分/举报概览）
43	GET	/users	JWT + ADMIN	用户列表（支持关键词搜索，分页，含佩戴徽章）
44	PUT	/users/{userId}/status	JWT + ADMIN	封禁/解封用户（status: 0/1）
45	GET	/demands	JWT + ADMIN	需求列表（支持 type/keyword/status 筛选，分页）
46	DELETE	/demands/{demandId}	JWT + ADMIN	硬删除需求（清理关联举报/积分流水引用）
47	GET	/reports	JWT + ADMIN	举报列表（按 status 筛选，分页）
48	PUT	/reports/{id}/resolve	JWT + ADMIN	处理举报（RESOLVED/DISMISSED，含可选操作）

3.4.10 举报模块—ReportController (/api/v1/reports)

#	方法	路径	认证	说明
49	POST	/	JWT	提交举报（目标: DEMAND/USER/MESSAGE, 5 种原因）

3.4.11 成就徽章模块—BadgeController (/api/v1/badges)

#	方法	路径	认证	说明
50	GET	/	JWT	获取全部 9 种徽章及当前用户进度
51	POST	/wear/{badgeKey}	JWT	佩戴已获得的徽章 (替换旧佩戴)
52	DELETE	/wear	JWT	取下当前佩戴的徽章 (幂等)
53	POST	/easter-egg	JWT	触发 EASTER_EGG 彩蛋徽章

3.4.12 WebSocket 实时通信

#	端点	协议	说明
54	/ws	STOMP over WS	WebSocket 握手端点
55	/topic/messages/{id}	STOMP 订阅	会话新消息实时推送
56	/topic/notifications/{id}	STOMP 订阅	系统通知实时推送
57	/app/chat.send	STOMP 发送	通过 WebSocket 发送消息

4. 数据库设计

4.1 核心表摘要

数据库采用 MariaDB/MySQL (utf8mb4 字符集), 共 15 张表, 通过 Flyway 版本化迁移管理 (V1-V14)。

表名	说明	关键索引	迁移版本
user	用户信息	uk_student_id (UNIQUE)	V1
privacy_profile	隐私配置	uk_privacy_user_id (UNIQUE, FK user)	V1
user_account	积分与信誉账户	uk_account_user_id (UNIQUE, FK user)	V1
demand	需求 (单表六类型)	idx_publisher, idx_acceptor, idx_type_status(type, status), idx_create_time	V2/V3/V7/V9
notification	系统通知	idx_user_read(user_id, is_read), idx_create_time	V4
evaluation	交易评价	uk_demand_evaluator(demand_id, evaluator_id) (UNIQUE)	V5
conversation	聊天会话	uk_demand_users(demand_id, user1_id, user2_id) (UNIQUE)	V6
message	聊天消息	idx_conv_time(conversation_id, create_time)	V6/V8
team_member	组队成员	uk_demand_user(demand_id, user_id) (UNIQUE)	V10
points_transaction	积分流水账本	idx_tx_user(user_id), idx_tx_time(create_time)	V11

表名	说明	关键索引	迁移版本
daily_checkin	每日签到	uk_user_date(user_id, checkin_date) (UNIQUE)	V11
user_favorite	需求收藏	uk_fav_demand_user (UNIQUE)	V12
report	举报记录	idx_report_target, idx_report_reporter, idx_report_status	V13
user_badge	已获得徽章	uk_user_badge (UNIQUE), idx_ub_user	V14
worn_badge	当前佩戴徽章	user_id (UNIQUE), idx_wb_user	V14

4.2 索引设计要点

- **demand.idx_type_status**: 首页需求列表按类型筛选 + 按状态筛选是最频繁的查询模式，复合索引避免回表
- **demand.idx_publisher/idx_acceptor**: 支持”我的发布/接单”查询，配合 Service 层批量加载消除 N+1
- **message.idx_conv_time**: 会话消息按时间升序分页加载，复合索引覆盖排序 + 过滤
- **notification.idx_user_read**: 未读通知数量是高频轮询查询（前端 10 秒轮询），复合索引直接命中
- **conversation.uk_demand_users**: 确定性 ID 排序 (user1_id < user2_id) 使该唯一索引能正确防止重复会话
- **evaluation.uk_demand_evaluator**: 保证每人在每个需求下仅能评价一次
- **daily_checkin.uk_user_date**: 签到唯一性约束 + 今日签到状态快速查询

4.3 隐私数据处理

- **密码**: 使用 BCrypt (strength=10) 哈希存储, user.password 列 VARCHAR(128)
 - **匿名发布**: demand.is_anonymous 控制需求级别的匿名, privacy_profile.is_anonymous + mask_name 控制用户级别的虚拟身份
 - **级联删除**: 核心外键均设置 ON DELETE CASCADE, 用户注销时自动清理关联数据
- 完整 ER 图（Mermaid 格式，含全部列定义和关系连线）请见 ER 图.md，建表 SQL 见 建表 SQL.sql。

5. 设计模式应用

5.0 与 v1.0 设计的差异说明

v1.0 详细设计文档中规划了**策略模式**（OrderMatchingStrategy □ DistanceMatchingStrategy / AIMatchingStrategy）和**观察者模式**（OrderStatusObserver □ AccountObserver / ChatObserver），但在实际工程实现中未采用。原因：

- **策略模式**: 需求类型的验证逻辑在当前阶段仅涉及 2-3 个字段检查，if/else if 分支足以清晰表达，引入策略接口 + 多实现 + 上下文工厂增加了不必要的间接层
- **观察者模式**: 订单状态变更仅需通知 2-3 个服务，直接同步调用比事件总线更直观，调试链路更短

以下为实际代码中采用的设计模式：

5.1 单表继承模式 (Single-Table Inheritance)

应用位置: 数据库层—demand 表设计

业务背景: 平台需支持六种互助需求类型 (跑腿、二手、组队、失物招领、学习互助、其他), 它们共享核心字段 (标题、描述、地点、报酬、状态), 但又有各自的特有属性 (跑腿的取件地点、组队的人数上限、失物招领的寻物/招领标记)。

模式应用: 所有类型共用一张 demand 表, type 列 (VARCHAR(32)) 作为类型鉴别器, attributes 列 (TEXT, JSON) 存储类型特有字段。在应用层 (DemandService.validateAttributes()) 按 type 分支校验 attributes 中的必填字段和值域。

使用理由: - 符合开闭原则 (数据库层面): 新增需求类型 (如 study、other) 无需 ALTER TABLE, 仅需在应用层扩展验证逻辑 - **MyBatis-Plus 兼容**: MyBatis-Plus 不支持 JPA 的 @Inheritance / @DiscriminatorColumn, 单表映射是自然选择 - **查询性能**: 需求广场统一列表无需跨 5+ 表 UNION, 单条 SELECT 完成

不使用该模式的后果: 类表继承设计 (base_order + paid_order + 5 张子表) 导致需求列表查询需 LEFT JOIN 或 UNION 多表, 随着类型增加查询计划持续恶化。

5.2 悲观锁模式 (Pessimistic Locking)

应用位置: PointsServiceImpl—freezeOnPublish() / unfreezeOnCancel() / transferOnComplete()

业务背景: 积分是平台的虚拟货币, 发布需求时冻结报酬积分, 取消时解冻退还, 完成时从发布者冻结余额转入接单者可用余额。在高并发场景下 (同一用户短时间内多次操作), 余额加减可能出现竞态条件。

模式应用: 所有积分写操作使用 SELECT ... FOR UPDATE 对 user_account 行加悲观锁, 保证同一用户的积分操作串行化执行。关键代码路径:

```
// PointsServiceImpl.freezeOnPublish()
UserAccount account = accountMapper.selectById(userId); // FOR UPDATE via wrapper
account.setAvailablePoints(account.getAvailablePoints() - amount);
account.setFrozenPoints(account.getFrozenPoints() + amount);
accountMapper.updateById(account);
```

使用理由: - **数据一致性**: 悲观锁是保证余额操作原子性的最可靠方式 (无需引入分布式锁中间件) - **简单可靠**: SELECT ... FOR UPDATE 是数据库原生支持的行锁机制, MySQL/MariaDB 开箱即用 - **校园场景 QPS 可控**: 用户量级下锁竞争概率极低, 悲观锁的性能开销可忽略

不使用该模式的后果: 基于乐观锁 (版本号) 的方案在高竞争场景下需要重试逻辑, 基于 CAS 的方案在复杂多表事务中实现难度高, 均不如悲观锁直接可靠。

5.3 批量加载模式 (Batch Loading)

应用位置: DemandServiceImpl.list() / myOrders() —批量预加载发布者信息

业务背景: 需求列表展示时需要显示发布者姓名和头像。若在渲染每条需求时逐条查询用户表 (N+1 查询模式), 30 条需求将产生 30 次额外数据库往返。

模式应用: Service 层在获取需求分页后, 收集所有 publisherId, 使用 userMapper.selectBatchIds(publisherIds) 一次性加载所有发布者信息, 构建 Map<Long, User> 缓存, 然后在组装 DemandResponse 时直接从缓存获取。

使用理由: - **消除 N+1 问题**: N 条需求仅需 2 次查询 (1 次查需求 + 1 次批量查用户), 而非 N+1 次 - **数据库连接友好**: 减少数据库连接池占用, 提升整体吞吐量

不使用该模式的后果： 首页加载 30 条需求将产生 31 次数据库查询，随用户量和需求量增长，响应延迟线性上升。

5.4 确定性 ID 排序模式

应用位置： ChatServiceImpl.getOrCreateConversation() — 会话创建时保证 user1_id < user2_id

业务背景： 两个用户在同一需求下只能有一个聊天会话。无论用户 A 先发起还是用户 B 先发起，数据库中应始终以同一对 (demand_id, user1_id, user2_id) 存储，以利用数据库 UNIQUE 约束防止重复。

模式应用： 创建会话前，应用层比较两个用户 ID，将较小者放入 user1_id、较大者放入 user2_id。数据库 UNIQUE(demand_id, user1_id, user2_id) 约束保证重复创建时抛出 DuplicateKeyException，由 Service 层捕获后查询已有会话返回。

使用理由： - **数据库约束兜底：** UNIQUE 索引是防重复的最可靠手段，不受应用层竞态影响 - **避免函数索引：** 相比 UNIQUE(demand_id, LEAST(a,b), GREATEST(a,b)) 的函数索引方案，应用层排序更简洁且跨数据库兼容 - **查询一致性：** 查找会话时同样应用确定性排序，一次查询命中

不使用该模式的后果： 需要使用应用层分布式锁防重，或在每次查询时尝试 (a,b) 和 (b,a) 两种组合，增加复杂度和出错风险。

6. AI 协作反思日志

本阶段使用 AI 辅助生成了类图初稿、API 文档、建表 SQL 初稿，并进行了 SOLID 原则审查。实际实现过程中，部分设计与初始规划产生了差异（如类表继承 □ 单表鉴别器、策略/观察者模式 □ 务实直接调用），这些工程决策均由团队根据实际开发体验做出。

- **反思日志文件：** AI 协作反思日志 #3.md
- **SOLID 检查清单：** SOLID 检查清单.md（记录了 AI 原始设计中的继承体系违规及修正方案）

详细反思内容（包括 AI 使用清单、误导分析、Prompt 改进等）请参见独立文件。

7. 交付物索引

序号	交付物	文件位置	状态
1	类图	类图.md	□ v2.1 — 基于实际 15 实体
2	SOLID 检查清单	SOLID 检查清单.md	□ v2.1 — 含实际实现差异分析
3	API 规范文档	API 规范文档.yaml	□ OpenAPI 3.0 格式
4	ER 图	ER 图.md	□ v2.1 — 基于实际 15 表 schema
5	建表 SQL	建表 SQL.sql	□ 可执行建表语句
6	AI 协作反思日志 #3	AI 协作反思日志 #3.md	□
7	详细设计文档（整合版）	本文档	□ v2.1（57 端点，15 表，V1-V14）